

分治

2025年5月19日 17:00

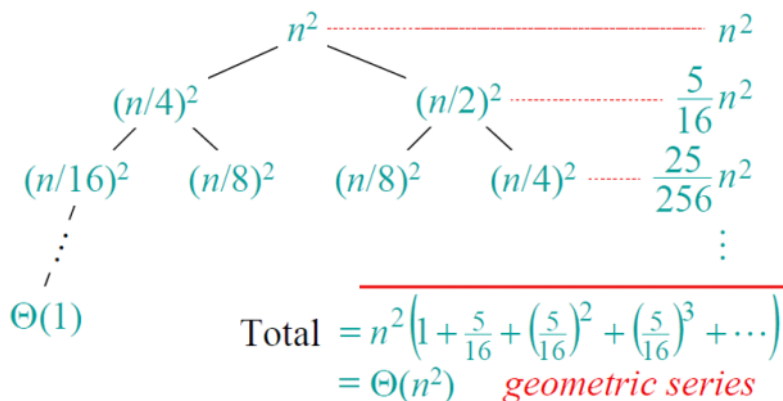
概念

分治法解决的问题的基本特征：

- 问题的规模缩小到一定程度就可以容易解决
- 问题可以分解为若干个规模较小的相同子问题
- 各个子问题是相互独立的
- 子问题的解可以合并为原问题的解

时间复杂度的分析方法：

Solve $T(n) = T(n/4) + T(n/2) + n^2$:



合并排序算法

■ 伪代码：

MERGE-SORT(A, low, high) //A[1..n]

if low < high then

mid = (low + high) / 2

MERGE-SORT(A, low, mid)

MERGE-SORT(A, mid+1, high)

MERGE(A, low, mid, high)

```
void merge(int A[], int low, int middle, int high){
    int i, j, k;
    int *B=new int[high-low+1]; // 临时存放合并结果
    i=low; j=middle+1; k=0;

    while(i<=middle&& j<=high) //两个子序列非空
        if(A[i]<=A[j]) B[k++]=A[i++];
        else B[k++]=A[j++];

    while (i<=middle) B[k++]=A[i++];
    while (j<=high) B[k++]=A[j++];

    for(i=low; i<=high; i++)
        A[i]=B[i]; // 搬回原来的数组
}
```

时间复杂度分析

归并排序的时间复杂度可以通过分析其递归结构来确定：

1. 递归部分：

- MERGE-SORT 函数将数组分成两半，并递归地排序两个子数组
- 这产生了递归方程 $T(n) = 2T(n/2) + f(n)$ ，其中 $f(n)$ 是合并操作的时间

2. 合并部分（MERGE函数）：

- while 循环比较两个子数组的元素并合并它们，这需要 $O(n)$ 时间
- 后面的两个 while 循环处理剩余元素，最多需要 $O(n)$ 时间
- 最后的 for 循环将临时数组复制回原数组，需要 $O(n)$ 时间
- 因此，合并操作 $f(n) = O(n)$

3. 总时间复杂度：

- 代入递归方程： $T(n) = 2T(n/2) + O(n)$
- 根据主定理或递归树分析，这个方程的解是 $T(n) = O(n \log n)$
- 这也证实了我们在第一个问题中看到的分析结果

归并排序在最好情况、平均情况和最坏情况下的时间复杂度都是 $O(n \log n)$ ，这是一个稳定的排序算法。

空间复杂度分析

1. 额外空间：

- MERGE 函数中创建了一个临时数组 B，大小为 $high-low+1$ ，即 $O(n)$
- 这是归并排序的主要空间开销

2. 递归栈空间：

- 递归调用会占用栈空间，递归深度为 $\log n$
- 因此递归栈空间为 $O(\log n)$

3. 总空间复杂度：

- 额外空间 + 递归栈空间 = $O(n) + O(\log n) = O(n)$

二分查找

■ 二分查找的伪代码（递归）：

BINARY-SEARCH-REC(A, low, high, x)

if low > high then

return -1

mid = (low + high) / 2

if x == A[mid] then

return mid

else if x > A[mid]

return BINARY-SEARCH-REC(A, mid+1, high, x)

else

return BINARY-SEARCH-REC(A, low, mid-1, x)

■ 二分查找的伪代码（非递归）：

BINARY-SEARCH-NOREC(A, n, x)

while low <= high

mid = (low + high) / 2

if x == A[mid]

then return mid

else if x > A[mid]

then low = mid+1

else

then high = mid-1

return -1

时间复杂性：

❖ 非递归：循环次数最大为 $\log n$ ，因此 $T(n) = O(\log n)$ 。

❖ 递归：
$$T(n) = \begin{cases} O(1) & n = 1 \\ T(n/2) + O(1) & n > 1 \end{cases}$$

基于后向导入或递归树可知： $T(n) = O(\log n)$

■ 空间复杂性:

❖ 非递归: $O(1)$ 。

❖ 递归: $O(\log n)$

循环日程赛

天数 编号	1	2	3	4	5	6	7
1	2	3	4	5	6	7	8
2	1	4	3	6	5	8	7
3	4	1	2	7	8	5	6
4	3	2	1	8	7	6	5
5	6	7	8	1	2	3	4
6	5	8	7	2	1	4	3
7	8	5	6	3	4	1	2
8	7	6	5	4	3	2	1

ROUND-ROBIN-CALENDAR-REC(A,n)

```
if n==1
  then A[1][1] = 1
return
```

ROUND-ROBIN-CALENDAR-REC(A,n/2)

COPY(n, A) //求解另一个规模为n/2的问题

COPY(n, A)

```
m = n/2;
for i = 1 to m
  for j = 1 to m
    do A[i][j+m] = A[i][j] + m; //小块的数值抄到右上角
       A[i+m][j] = A[i][j+m]; //右上抄到左下
       A[i+m][j+m] = A[i][j]; //左上抄到右下
```

■ 时间复杂性:

❖ 递归

$$T(n) = \begin{cases} O(1) & n=1 \\ T(n/2) + c(n/2)^2 & n>1 \end{cases}$$

✓ 基于后向导入或递归树可知: $T(n)=O(n^2)$ 。

■ 空间复杂性:

❖ 递归: $O(\log n)$ 。

快速排序

第一次划分 PARTITION(A, 0, 6)

1. 选择 pivot = A[0] = 49

2. 初始 leftwall = 0

3. 开始遍历 j = 1 到 6:

- j = 1: A[1] = 38 < pivot(49)
 - leftwall++ 变为 1
 - 交换 A[1] 和 A[1] (实际上没变)
 - 数组保持 [49, 38, 65, 97, 76, 13, 27]
- j = 2: A[2] = 65 > pivot(49)
 - 不做任何操作
 - 数组保持 [49, 38, 65, 97, 76, 13, 27]
- j = 3: A[3] = 97 > pivot(49)
 - 不做任何操作
 - 数组保持 [49, 38, 65, 97, 76, 13, 27]
- j = 4: A[4] = 76 > pivot(49)
 - 不做任何操作
 - 数组保持 [49, 38, 65, 97, 76, 13, 27]
- j = 5: A[5] = 13 < pivot(49)
 - leftwall++ 变为 2
 - 交换 A[2] 和 A[5]
 - 数组变为 [49, 38, 13, 97, 76, 65, 27]
- j = 6: A[6] = 27 < pivot(49)
 - leftwall++ 变为 3
 - 交换 A[3] 和 A[6]
 - 数组变为 [49, 38, 13, 27, 76, 65, 97]

4. 遍历结束, 交换 A[0] 和 A[leftwall]

- 交换 A[0] 和 A[3]
- 数组变为 [27, 38, 13, 49, 76, 65, 97]
- pivotpos = 3

5. 返回 pivotpos = 3

此时, pivotpos = 3, 基准元素 49 已经放到了正确的位置, 小于 49 的元素 (27, 38, 13) 在左边, 大于 49 的元素 (76, 65, 97) 在右边。

```

QUICK-SORT(A, low, high)
    if low < high
        then pivotpos = PARTITION(A, low, high) //划分序列
            QUICK-SORT(A, low, pivotpos-1)      //对左区间递归排序
            QUICK-SORT(A, pivotpos+1, high)      //对右区间递归排序

PARTITION(A, low, high)
    pivot = A[low] //选择第一个元素作为基准
    leftwall = low //左墙初始位置

    for j = low+1 to high
        if A[j] < pivot
            then leftwall = leftwall + 1
                SWAP(A[leftwall], A[j]) //将小于基准的元素交换到左区

    SWAP(A[low], A[leftwall]) //将基准元素放到最终位置
    return leftwall //返回基准元素的位置

```

■ 时间复杂性 - 最坏情况： $O(n^2)$

$$T(n) = \begin{cases} O(1) & n = 1 \\ T(n-1) + O(n) & n > 1 \end{cases}$$

■ 时间复杂性 - 最好情况： $O(n \log n)$

$$T(n) = \begin{cases} O(1) & n = 1 \\ 2T(n/2) + O(n) & n > 1 \end{cases}$$

■ 时间复杂性 - 平均情况： $O(n \log n)$

$$T(n) = 1/n \sum_{k=1}^n (T(n-k) + T(k-1)) + n$$

■ 空间复杂性：

❖ 最坏： $O(n)$ ；最好： $O(\log n)$ ；平均： $O(\log n)$

线性时间选择

```

function select(A, k):
    if len(A) <= 5:
        sort A
        return A[k-1] // 第 k 小的元素（注意索引从 0 开始）

// 1. 将 A 分组（每组 5 个元素）
divide A into groups of at most 5 elements

// 2. 找出每组的中位数
for each group:
    sort group
    find and store the median into medians[]

// 3. 递归地找出中位数的中位数
pivot = select(medians, len(medians) // 2)

// 4. 用 pivot 划分 A 为三部分
L = [x in A | x < pivot]
E = [x in A | x == pivot]
G = [x in A | x > pivot]

// 5. 根据 k 决定递归方向
if k <= len(L):
    return select(L, k)
else if k <= len(L) + len(E):
    return pivot
else:
    return select(G, k - len(L) - len(E))

```

棋盘覆盖问题

非叶节点代价:

- 第1层: c
- 第2层: $4c$
- 第3层: 4^2c
- ...
- 第 k 层: $4^{k-1}c$

非叶节点总代价 = $c + 4c + 4^2c + \dots + 4^{k-1}c$

这是一个等比数列, 首项为 c , 公比为 4 , 项数为 k 。根据等比数列求和公式: $S_n = \frac{a_1(1-q^n)}{1-q}$, 其中 a_1 是首项, q 是公比。

所以非叶节点总代价 = $c \cdot \frac{1-4^k}{1-4} = \frac{c}{3}(4^k - 1) = O(4^k)$

■ 时间复杂性:

❖ 递归:

$$T(k) = \begin{cases} O(1) & k = 0 \\ 4T(k-1) + O(1) & k > 0 \end{cases}$$

✓ 基于后向导入或递归树可知: $T(k) = O(4^k)$ 。

■ 空间复杂性:

❖ 递归: $O(k)$ 。

其他问题
大数据乘法

大数据乘法

$X =$

a

b

■ 分治法（二分）：

$Y =$

c

d

❖ 每个迭代分解为3次相乘：

$$XY = ac \cdot 2^n + ((a-b)(d-c) + ac + bd) \cdot 2^{n/2} + bd \quad \text{或}$$

$$XY = ac \cdot 2^n + ((a+b)(d+c) - ac - bd) \cdot 2^{n/2} + bd$$

时间复杂性分析：

$$T(n) = \begin{cases} O(1) & n = 1 \\ 3T(n/2) + O(n) & n > 1 \end{cases} \quad T(n) = O(n^{\log_3}) \quad \checkmark$$

空间复杂性分析： $S(n) = O(\log n)$

Karatsuba的关键洞察是：不需要分别计算 $a \times d$ 和 $b \times c$ ，可以通过以下方式：

1. 计算 $z_1 = a \times c$
2. 计算 $z_3 = b \times d$
3. 计算 $z_2 = (a+b) \times (c+d)$

然后注意到： $z_2 = (a+b) \times (c+d) = a \times c + a \times d + b \times c + b \times d = z_1 + a \times d + b \times c + z_3$

因此，我们可以得到： $a \times d + b \times c = z_2 - z_1 - z_3$

所以，最终的乘积可以表示为： $XY = z_1 \times 2^n + (z_2 - z_1 - z_3) \times 2^{(n/2)} + z_3$

动态规划

2025年5月21日 10:51

概念:

动态规划的基本要素

最优子结构性性质 (一个问题的最优解包含其子问题的最优解)

子问题重叠性质 (不是必要条件)

自底向上求解, 先填停止条件, 求解每一级子问题并保存, 直至得到原问题的解

动态规划步骤

分析最优解性质, 考察是否适合采用动态规划方法

建立递归公式和动态规划方程 (包括停止条件和递归式)

自底向上计算最优值并记录相关信息

构造最优解

备忘录方法为动态规划方法的变形

相同点:

采用表格保存已解决的子问题的值, 在下次需要时直接查表而无需重新计算

不同点:

备忘录方法采用自顶向下的递归方式, 而动态规划方法 则采用自底向上的递归方式

其控制结构与直接递归的控制结构相同, 区别在于备忘录方法为每个已解的子问题建立备忘录以备需要时看, 避免了相同子问题的重复求解

斐波那契数列

FIB2(n, F)

$F[0] = 0$

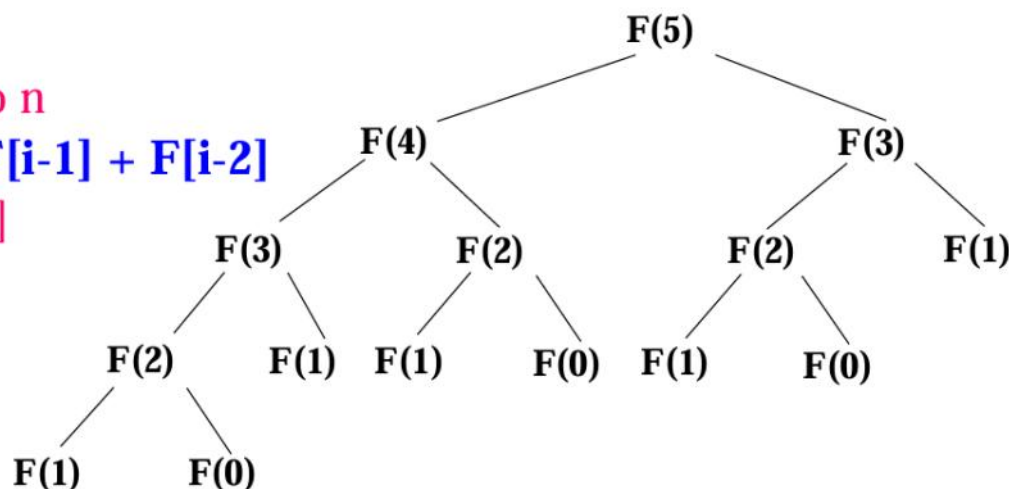
$F[1] = 1$

for i = 2 to n

$F[i] = F[i-1] + F[i-2]$

return $F[n]$

自底向上并保存子问题值的求解:



保存子问题值留作后面查找使用。其时间复杂性为 $O(n)$, 空间复杂性为 $O(n)$ 。

矩阵连乘动态规划版本

$$m[i, j] = \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{m[i, k] + m[k+1, j] + p_{i-1}p_kp_j\} & i < j \end{cases}$$

■ 采用DP求解矩阵连乘问题的伪代码：

```

MATRIX-CHAIN-ORDER(n, p) //p[1..n+1]为n个矩阵的维数,
    //m[1..n, 1..n]为最优值, s[1..n, 1..n]为最优决策
    for i = 1 to n
        m[i,i] = 0
    for l = 2 to n // l为链的长度
        for i = 0 to n-l+1
            j = i+l-1 //长度为l的链A[i,j]
            m[i, j] = ∞
            for k = i to j-1 //求解链A[i,j]的最优值m[i,j]以及最优决策s
                q = m[i,k] + m[k+1,j] + pi-1pkpj
                if q < m[i,j]
                    m[i,j] = q
                    s[i,j] = k
    return m and s

```

时间复杂度：三重循环，每层循环的规模都不超过**O(n)**，总的嵌套循环执行**O(n³)**，**最内层循环体计算为2次加法，2次乘法是常数时间。**

因此总的**时间复杂度为O(n³)**。

空间复杂度为O(n²)

■ 构造矩阵连乘问题最优解的伪代码:

PRINT-OPTIMAL-PARENS(s, i, j)

if $i == j$

then print “A[”+ i + “]”

else print “(”

PRINT-OPTIMAL-PARENS(s, i, **s[i,j]**)

PRINT-OPTIMAL-PARENS(s, **s[i,j]+1**, j)

print “)”

❖ 时间复杂性:

✓ $O(n)$

❖ 空间复杂性:

✓ $O(n)$

? 不挂

矩阵连乘备忘录方法

MEMORIZED-MATRIX-CHAIN(p, n)

//m[1..n, 1..n] 为最优值

for i = 1 to n

for j = i to n

if $i == j$

$m[i, j] = 0$

else

$m[i, j] = \infty$

return LOOKUP-CHAIN(m, p, 1, n)

LOOKUP-CHAIN(m, p, i, j)

if $i == j$

return 0;

if $m[i, j] < \infty$

return $m[i, j]$

for $k = i$ to $j-1$

$q = \text{LOOK-CHAIN}(m, p, i, k)$

+ $\text{LOOK-CHAIN}(m, p, k+1, j)$

+ $p_{i-1}p_kp_j$

if $q < m[i, j]$

$m[i, j] = q$

$s[i, j] = k$

return m and s

❖时间复杂性

✓ $O(n^3)$, 总共 n^2 个记录项, 每个记录项填充
花时间约 $O(n)$

❖空间复杂性:

✓ $O(n^2)$

最长公共子序列 动态规划法

$$c[i][j] = \begin{cases} 0 & i = 0, j = 0 \\ c[i-1][j-1] + 1 & i, j > 0; x_i = y_j \\ \max\{c[i][j-1], c[i-1][j]\} & i, j > 0; x_i \neq y_j \end{cases}$$

LCS(X,Y)

m = length(X)

n = length(Y)

// 初始化表格边界

for i = 1 to m

c[i][0] = 0

for j = 1 to n

c[0][j] = 0

// 填充DP表格

for i = 1 to m

for j = 1 to n

if X[i] == Y[j] then

c[i][j] = c[i-1][j-1] + 1

b[i][j] = 1 // 对角线方向 (↖)

else if c[i-1][j] >= c[i][j-1]

c[i][j] = c[i-1][j]

b[i][j] = 3 // 向上方向 (↑)

else

c[i][j] = c[i][j-1]

b[i][j] = 2 // 向左方向 (←)

// 算法时间复杂度为 $O(mn)$ => 若 $m \approx n$, 则为 $O(n^2)$

LCS-PRINT(b, X, i, j)

if i = 0 or j = 0

then return

if b[i][j] = 1

```

        then return
    if b[i][j] = 1
        then LCS-PRINT(b, X, i-1, j-1)
            print X[i]
    else if b[i][j] = 3
        then LCS-PRINT(b, X, i-1, j)
    else LCS-PRINT(b, X, i, j-1)    //b[i][j] = 2

```

最长公共子序列备忘录直接递归法（课件上的代码有问题）

// 同时记录长度和决策

```

function LCS-REC-MEMO-WITH-DECISION(X, Y, i, j, memo, decisions)
    if memo[i][j] is not computed
        if i = 0 or j = 0
            memo[i][j] = 0
        else if X[i] == Y[j]
            memo[i][j] = LCS-REC-MEMO-WITH-DECISION(X, Y, i-1, j-1, memo, decisions) + 1
            decisions[i][j] = "diagonal" // 表示匹配，选择了对角线方向
        else
            left = LCS-REC-MEMO-WITH-DECISION(X, Y, i, j-1, memo, decisions)
            up = LCS-REC-MEMO-WITH-DECISION(X, Y, i-1, j, memo, decisions)

            if up >= left
                memo[i][j] = up
                decisions[i][j] = "up" // 表示选择了上方
            else
                memo[i][j] = left
                decisions[i][j] = "left" // 表示选择了左方

    return memo[i][j]

```

01背包问题 动态规划版

约束条件：

$$\begin{cases} \sum_{i=1}^n w_i x_i \leq W \\ x_i \in \{0,1\}, (1 \leq i \leq n) \end{cases}$$

目标函数：

$$\max \sum_{i=1}^n v_i x_i$$

$$C[0][j]=C[i][0]=0$$

$$C[i][j]=\begin{cases} C[i-1][j] & j < w_i \\ \max\{C[i-1][j], C[i-1][j-w_i]+v_i\} & j \geq w_i \end{cases}$$

KNAPSACK-01(w, v, W) //w为重量, v为价值, W为容量
//C[1..n, 1..n]为最优值

for i = 1 to n

do $C[i, 0] = 0$

for j = 1 to W

do $C[0, j] = 0$

for i = 1 to n

for j = 1 to W

if $j < w[i]$

then $C[i, j] = C[i-1][j]$

else

then $C[i, j] = \max\{C[i-1][j], C[i-1][j-w[i]] + v[i]\}$

return C

❖时间复杂性:

✓ $O(nW)$

❖空间复杂性:

✓ $O(nW)$

TRACEBACK-01(w, W, C) //w为重量, W为容量, C为最优值 //x[1..n]为构建的最优解

n = w.length - 1

j = W //Remained Capacity

for i = n to 1

do if $C[i][j] == C[i-1][j]$ then

x[i] = 0

else

x[i] = 1

j -= w[i]

return x

❖时间复杂性:

✓ $O(n)$

❖空间复杂性:

✓ $O(n)$

■ 示例: $w = (2, 2, 6, 5, 4)$, $v = (6, 3, 5, 4, 6)$, $W = 10$

i \ j	0	1	2	3	4	5	6	7	8	9	10
0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	6	6	6	6	6	6	6	6	6
2	0	0	6	6	9	9	9	9	9	9	9
3	0	0	6	6	9	9	9	9	11	11	14
4	0	0	6	6	9	9	9	10	11	13	14
5	0	0	6	6	9	9	12	12	15	15	15

❖ 最优解: $X = (x_1, x_2, x_3, x_4, x_5) = (1, 1, 0, 0, 1)$

背包问题 跳跃点法

由此, 可计算出 $p[i]$:

✓ $p[i] = p[i-1] \cup q[i-1]$

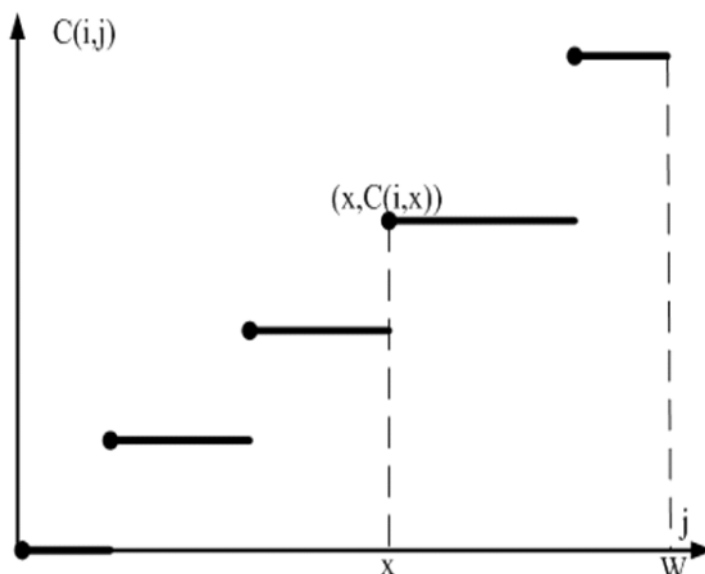
$q[i-1]$ 可由 $p[i-1]$ 计算出:

✓ $q[i-1] = p[i-1] \oplus (w_i, v_i)$

求解过程如下:

• 初始时:

✓ $p[0] = \{(0,0)\}$



■ $w = (2, 2, 6, 5, 4), v = (6, 3, 5, 4, 6), W = 10。$

$$q[0] = p[0] \oplus (w_1, v_1) = \{(2, 6)\}$$

$$p[1] = p[0] \cup q[0] = \{(0, 0), (2, 6)\}$$

$$p[0] = \{(0, 0)\}$$

$$p[i] = p[i-1] \cup q[i-1]$$

$$q[i-1] = p[i-1] \oplus (w_i, v_i)$$

$$q[1] = p[1] \oplus (w_2, v_2) = \{(2, 3), (4, 9)\}$$

$$p[1] \cup q[1] = \{(0, 0), (2, 6), (2, 3), (4, 9)\}$$

$$p[2] = \{(0, 0), (2, 6), (4, 9)\}$$

$$q[2] = p[2] \oplus (w_3, v_3) = \{(6, 5), (8, 11), (10, 14)\}$$

$$p[2] \cup q[2] = \{(0, 0), (2, 6), (4, 9), (6, 5), (8, 11), (10, 14)\}$$

$$p[3] = \{(0, 0), (2, 6), (4, 9), (8, 11), (10, 14)\}$$

.....

■ $w = (2, 2, 6, 5, 4), v = (6, 3, 5, 4, 6), W = 10。$

$$p[0] = \{(0, 0)\}$$

$$p[1] = \{(0, 0), (2, 6)\}$$

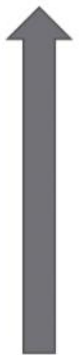
$$p[2] = \{(0, 0), (2, 6), (4, 9)\}$$

$$p[3] = \{(0, 0), (2, 6), (4, 9), (8, 11), (10, 14)\}$$

$$p[4] = \{(0, 0), (2, 6), (4, 9), (7, 10), (8, 11), (9, 13), (10, 14)\}$$

$$p[5] = \{(0, 0), (2, 6), (4, 9), (6, 12), (8, 15)\}$$

构造最优解: $(1, 1, 0, 0, 1)$



构造解的步骤

步骤1: 初始化 $i = n$, j 初始化为 $p[n]$ 中的最大值, m 初始化为 $p[n]$ 中的最大价值。

步骤2: 检查 $p[i-1]$ 中的所有点 (w, v) , 如果 $w + w_i = j$ 并且 $v + v_i = m$, 则 $x_i = 1, j = w, m = v$; 否则 $x_i = 0$ 。 $i--$ 。重复步骤2直到 $i = 0$ 为止。

KNAPSACK-01-OPT(w, v, W)

```
//w为重量, v为价值, W为容量
//p[1..n, 2]: 存放指定i的跳跃点即(j, C[i][j])值对。
//q[1..n, 2]: 如定义
p[0] = {(0, 0)}
for i = 1 to n
     $q[i-1] = p[i-1] \oplus (w[i], v[i])$  //最多有 $2^{i-1}$ 个跳跃点
     $p[i] = p[i-1] \cup q[i-1]$  //最多 $2^i$ 个跳跃点
    PROCESS(p[i], W) //剔除受控/无效的跳跃点
return p
```

当每个物品的重量 w_i 是整数时, 循环体中跳跃点个数最多是 W 个, 时间复杂度是 $O(nW)$

❖ 时间复杂性:
✓ $O(\min(nW, 2^n))$
❖ 空间复杂性:
✓ $O(n)$

BACKTRACK-OPT(w, v, p, n)

```
//w为重量, v为价值, p为跳跃点的值对 //x[1..n]: 存放问题的解
j = max(p[n][0]);
m = max(p[n][1]);
for i = n to 1
    if (j, m) ==  $p[i-1] \oplus (w[i], v[i])$ 
         $x[i] = 1$ ;
         $j = p[i-1][0]$ ;
         $m = p[i-1][1]$ ;
    else
         $x[i] = 0$ 
return x
```

❖ 时间复杂性:
✓ $O(\min(nW, 2^n))$
❖ 空间复杂性:
✓ $O(n)$

约束条件：

$$\begin{cases} \sum_{i=1}^n w_i x_i \leq W \\ x_i \in [0, 1.0], (1 \leq i \leq n) \end{cases}$$

目标函数：

$$\max \sum_{i=1}^n v_i x_i$$

GREEDY-KNAPSACK(w, v, W) //w为重量,v为价值, W为容量

SORT-ASC-BY-V(v,w) //按照物品的单位价值非降序排序, 同时调整v和w

n = w.length

//x[1..n]为最优值, 存放已选物品的相应重量。各元素初始化为0

wRemained = W

for i = 1 to n

 if wRemained <= 0

 break

 else if **wRemained >= w[i]**

 x[i] = w[i]

 wRemained = wRemained - w[i]

 else

 x[i] = wRemained

 break

return x

❖时间复杂性:

✓ **O(nlogn)**

❖空间复杂性:

✓ **O(1)**

贪心

2025年5月21日 18:33

概念

最优子结构性质（一个问题的最优解包含其子问题的最优解）

贪心选择性质（通过一系列的逐步局部最优选择使得最终的选择方案是全局最优的）

贪心选择可以依赖于以往的选择, 但不依赖于将来 所作出的选择, 也不依赖于子问题的解

动态规划采用自底向上的方式解各子问题, 而贪心 算法采用自顶向下的方式

会场安排问题

GREEDY-ACTIVITY-SELECTOR(B,E)

SORT-ASC-BY-F(B, E) //对E按减序排序并同时调整B

$n = B.length$

$A = \{1\}$ //首先选择会议1

$k = 1$ //已被选择会议集合中 最晚结束的会议(k)

for $m = 2$ to n

if $B[m] \geq E[k]$ //会议m的开始时间不小于会议k的结束时间

then $A = A \cup \{m\}$ //将会议m加入到集合A中

$k = m$ //此时集合A中最晚结束的会议为m

return A

求解过程（假设先已按照结束时间的非减序排序）：

会议 i	1	2	3	4	5	6	7	8	9	10	11
开始时间 b_i	1	3	0	5	3	5	6	8	8	2	12
结束时间 e_i	4	5	6	7	8	9	10	11	12	13	14

✓ 一个最优解为：{1, 4, 8, 11}。

时间复杂性：

❖ (a): $O(n \log n)$, (b): $O(n)$ 。 $\rightarrow O(n \log n)$
(c) : $n \rightarrow O(n)$

空间复杂度

❖ (a): $O(m \log n)$, (b): $O(n)$ 。 $\rightarrow O(m \log n)$
 (c) : $n \rightarrow O(n)$

空间复杂性:

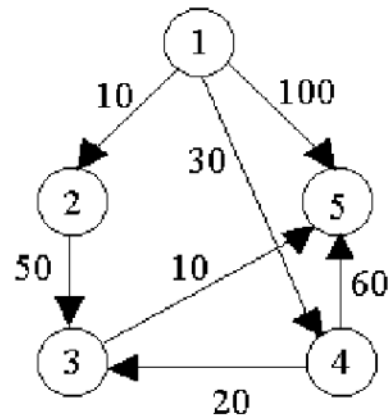
❖ (a): $O(\log n)$, (b): $O(1)$ 。 $\rightarrow O(\log n)$

单源最短路径问题SSSP

迪杰斯特拉算法

迭代	S	u	dist[2]	dist[3]	dist[4]	dist[5]
初始	{1}	-	10	maxint	30	100
1	{1,2}	2	10	60	30	100
2	{1,2,4}	4	10	50	30	90
3	{1,2,4,3}	3	10	50	30	60
4	{1,2,4,3,5}	5	10	50	30	60

	1	2	3	4	5
pre	-	1	4	1	3



DIJKSTRA(G, w, s)

INITIALIZE-SINGLE-SOURCE(G, s)

$S = \Phi$

$Q = G.V$

while $Q \neq \Phi$

$u = \text{EXTRACT-MIN}(Q)$

$S = S \cup \{u\}$

for each $v \in G.\text{Adj}[u]$

RELAX(u, v, w)

$d[s] = 0$
 for each $v \in G.V - \{s\}$
 $d[v] = \infty$

if $d[v] > d[u] + w(u, v)$
 $d[v] = d[u] + w(u, v)$

时间复杂性:

- ❖ EXTRACT-MIN() 的时间复杂性为 $O(\log n)$;
- ❖ 二重循环的执行次数为 $(n-1)+(n-2)+\dots+1 = n(n-1)/2$ ，即时间复杂性为 $O(n^2)$ 。
- ❖ 所以，该算法的时间复杂性为 $O(n^2)$ 。

空间复杂性:

- ❖ 优先队列Q的大小为 $n-1$;
- ❖ 所以，该算法的空间复杂性为 $O(n)$ 。

1. 初始: $S=\{1\}$, $\text{dist}=[0,10,\text{maxint},30,100]$, $\text{pre}=[-,1,-,1,1]$
2. 第1轮: 选择节点2
 - $S=\{1,2\}$, $u=2$
 - 更新: $\text{dist}[3]=60$, $\text{pre}[3]=2$
 - 此时: $\text{dist}=[0,10,60,30,100]$
3. 第2轮: 选择节点4
 - $S=\{1,2,4\}$, $u=4$
 - 更新: $\text{dist}[3]=50$, $\text{pre}[3]=4$ (因为 $10+50 > 30+20$)
 - 更新: $\text{dist}[5]=90$, $\text{pre}[5]=4$
 - 此时: $\text{dist}=[0,10,50,30,90]$
4. 第3轮: 选择节点3
 - $S=\{1,2,4,3\}$, $u=3$
 - 检查节点5: $\text{dist}[5] > \text{dist}[3] + w(3,5)$? $90 > 50 + 10 = 60$? 是的
 - 更新: $\text{dist}[5]=60$, $\text{pre}[5]=3$
 - 此时: $\text{dist}=[0,10,50,30,60]$
5. 第4轮: 选择节点5
 - $S=\{1,2,4,3,5\}$, $u=5$
 - 算法结束

最小生成树问题

Prim算法

```

MST-PRIM( $G, w, r$ ) //  $r$ 为根结点
  for each  $u \in G.V$  //初始化
     $u.key = \infty$  //  $u.key$ 为连接 $u$ 和树中结点所有边中最小边的权重
     $u.p = NIL$  //  $u.p$ 为结点 $u$ 的父结点
   $r.key = 0$ 
   $Q = G.V$  // 优先队列。后述的 $U = V - Q$ 
  while  $Q \neq \Phi$ 
     $u = \text{EXTRACT-MIN}(Q)$ 
    for each  $v \in G.Adj[u]$ 
      if ( $v \in Q$  and  $w(u, v) < v.key$ )
         $v.p = u$ 
         $v.key = w(u, v)$  // DECREASE-KEY

```

Prim算法时间复杂性:

❖ $O(n^2)$, 这里 n 为图中结点数。

Prim算法空间复杂性:

❖ $O(n)$ 。

1. 初始化:

- 所有节点的key值设为无穷大, 父节点设为NIL
- 选择的起点 r 的key值设为0
- 将所有节点放入优先队列 Q

2. 主循环:

- 每次从 Q 中取出key值最小的节点 u
- 对 u 的每个邻居 v 检查:
 - 如果 v 还在 Q 中, 且边 (u,v) 的权重小于 v 当前的key值
 - 则更新 v 的父节点为 u , key值为边 (u,v) 的权重

3. 算法结束: 当 Q 为空时, 算法结束, 所有节点的父节点关系构成最小生成树

回溯法

2025年5月21日 23:09

概念

基于深度优先策略（能进则进，不进则换，不换则退）

解空间是指问题所有可能解的集合。这些解包括了可行解和不可行解

解空间树是将整个解空间组织成树形结构的一种方式，便于系统地枚举和搜索所有可能的解

扩展结点:一个正在生成孩子的结点称为扩展结点

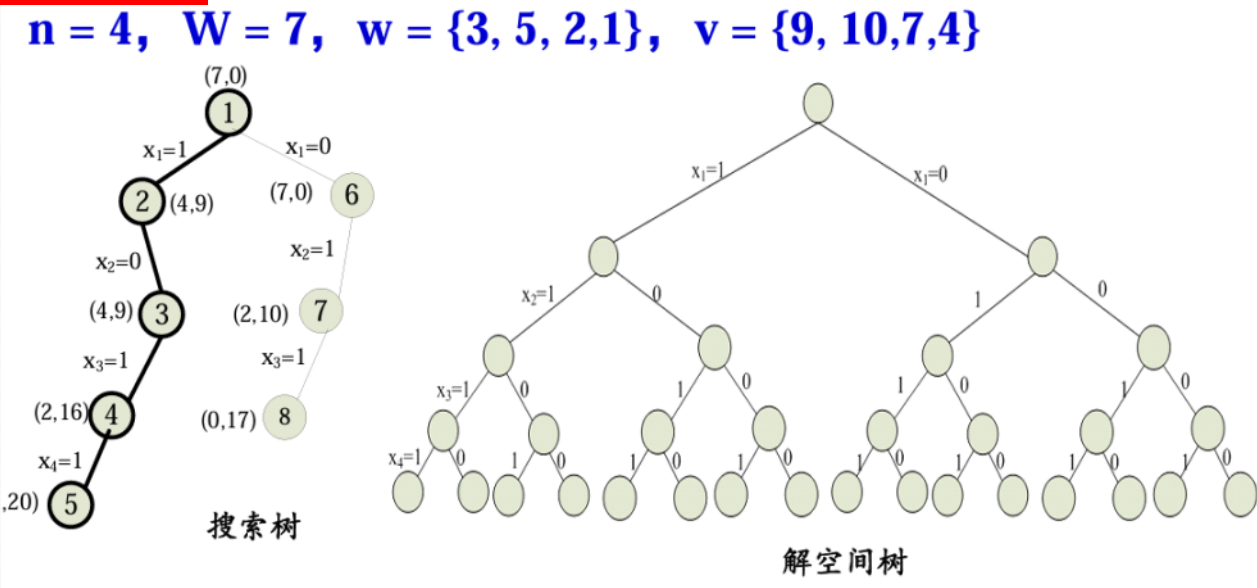
活结点:一个自身已生成但其孩子还没有全部生成的节点称做活结点

死结点:一个所有孩子已经产生的结点称做死结点

搜索树: 搜索过程中动态形成的树

树类型	结构形式	分支数	代表问题类型	典型问题	
子集树	二叉树	2	子集选择问题（选或不选）	0-1 背包、子集和问题	
排序树	n 层排列树	n-i（动态）	排列问题	旅行商、全排列、八皇后	
满 m 叉树	m 叉树	固定 m	多项选择、状态树搜索	图染色、博弈搜索、资源分配	

01背包问题回溯法



思考：能否通过更改隐约束 $cp+rp>bestp$ 来提高搜索效率？

- rp 表示第 $t+1$ 个物品到第 n 个物品的总价值。
- 剩余容量所能容纳的从第 $t+1$ 个物品到第 n 个物品的最大价值（用 brp 表示）肯定小于或等于 rp ，用 brp 取代 rp
- 限界条件可改写为： $cp+brp > bestp$ 。

rp 是宽松上界， brp 是更紧的上界

rp 不考虑剩余容量，可能高估了能取得的价值 → 导致保留了很多“无用路径”

brp 是考虑了剩余容量的限制后，能得到的最大价值 → 更接近真实情况

所以，用 $cp + brp > bestp$ 可以剪掉更多无法超过 $bestp$ 的分支，提高剪枝效率

brp 的求法是按序号一直取，然后最后一个尽量取

```

// 0-1 背包问题的递归式回溯伪代码
backtrack-Knapsack-01 (t, w, v, W)
    // t: 当前处理的物品索引
    // w: 物品重量数组
    // v: 物品价值数组
    // W: 背包总容量
    // cw: 当前背包中物品的总重量
    // cp: 当前背包中物品的总价值
    // bestp: 当前找到的最优解价值
    // x: 解向量, x[i]表示是否选择第i个物品

    if t > n                                // 到达叶子结点
        OUTPUT(x)                          // x[1..n]: 为解向量, x[i]取值0或1
        bestp = cp
        return

    if cw + w[t] <= W                        // 搜索左子树 (选择第t个物品)
        x[t] = 1
        cw += w[t]                          // 当前背包中物品的总重量
        cp += v[t]                          // 当前背包中物品的总价值
        backtrack-knapsack-01(t+1, w, v, W)
        cw -= w[t]                          // 回溯, 恢复状态
        cp -= v[t]                          // 回溯, 恢复状态

    if bound(t+1, W-cw, cp, w, v) > bestp    // 搜索右子树 (不选择第t个物品)
        x[t] = 0
        backtrack-knapsack-01(t+1, w, v, W)

// 计算上界函数
bound(i, cleft, cp, w, v)    // i为第i层, cleft为背包的剩余容量, cp为当前背包中物品的总价值
    b = cp
    while i <= n && w[i] <= cleft
        cleft -= w[i]
        b += v[i]
        i++

    if i <= n // 装满背包
        b += v[i] / w[i] * cleft

    return b

```

■判断约束函数需 $O(1)$ ，在最坏情况下有 2^n-1 个左孩子，约束函数耗时最坏为 $O(2^n)$ 。

■计算上界限函数需要 $O(n)$ 时间，在最坏情况下有 2^n-1 个右孩子需要计算上界，界限函数耗时最坏为 $O(n2^n)$ 。

■时间复杂度： $O(2^n) + O(n2^n) = O(n2^n)$ 。

■空间复杂度： $O(n)$ ，即递归深度。

作业调度问题

■批处理作业调度问题描述：

- ❖ 给定 n 个作业的集合 $\{J_1, J_2, \dots, J_n\}$ 。每个作业须先由机器1处理，然后由机器2处理。
- ❖ 作业 J_i 需要机器 m 的处理时间为 $t_{mi}(m=1, 2)$ ；对于一个确定的作业调度，设 F_{mi} 是作业 i 在机器 m 上完成处理的时间。
- ❖ 所有作业在机器2上完成处理的时间和，称为该作业调度的完成时间和 f ， $f = \sum F_{2i}$ 。

■批处理作业调度的目标：

- ❖ 找出使得 f 最小的调度方案。

解空间的组织结构：

- 排列树，深度为问题的规模（即 n ）。

- 约束条件：无；
- 限界条件：
 - ✓ 假设 **cf** 表示当前已调度的作业在 **机器2** 上的完成时间和，**bestf** 表示当前最优调度所需要的完成时间和： **$cf < bestf$** 。



```

class Flowshop {
private:
    void Backtrack(int i);
    int **M, // 各作业所需的处理时间
        *x, // 当前作业调度
        *bestx, // 当前最优作业调度
        *f2, // 机器2完成处理时间
        f1, // 机器1完成处理时间
        f, // 完成时间和
        bestf, // 当前最优完成时间和
        n; // 作业数
};

void backtrack(int i)
{
    if i > n {
        for (j = 1; j <= n; j++)
            bestx[j] = x[j]; //初始时x[i]=i
        bestf = f;
        return;
    }
}

```

```

for (j = i; j <= n; j++) {
    f1+=M[x[j]][1]; //机器1完成处理时间
    f2[i] = ((f2[i-1]>f1) ? f2[i-1] : f1) + M[x[j]][2];
    f += f2[i];
    if (f < bestf) {
        Swap(x[i], x[j]);
        backtrack(i+1);
        Swap(x[i], x[j]);
    }
    f1 -= M[x[j]][1];
    f -= f2[i];
}
}

```

Backtrack在每一个结点处限界函数耗费 $O(1)$ 计算时间，需要判断限界函数的结点在最坏的情况下有 $1+n+n(n-1)+n(n-1)(n-2)+\dots+n(n-1)+\dots+2 \leq nn!$ 个，故耗时 $O(nn!)$;

叶子结点处记录当前最优解耗时为 $O(n)$,叶子结点最多有 $n!$ 个，故耗时为 $O(nn!)$.

整个算法的计算时间复杂度为

$O(nn!) + O(nn!) = O(nn!) = O((n+1)!)$

空间复杂度 $O(n)$

n皇后问题满m叉树版本

问题解的形式：表示为n元组 $(x_1, x_2, \dots, x_n)$ 的形式，其中 $x_i (i=1, 2, \dots, n)$ 表示第i个皇后放置在**第i行第 x_i 列**的位置；

显式约束：**n个皇后不同行**；

隐式约束：n个皇后**不同列**而且不在**正反对角线**上；

解空间：根据显式约束，**第i(i=1, 2, ..., n)个皇后有n个位置**可供选择，即**第i行的n个列**位置，即 $x_i \in \{1, 2, \dots, n\}$ 。

很显然满足显式约束的n元组共有 **n^n** 种，它构成n皇后问题的解空间。→ 满**m(m=n)**叉树

n皇后问题（一）的递归式回溯伪代码：

BACKTRACK-NQUEEN1-REC(t)

if $t > n$

 OUTPUT(x)

else

 for $i = 1$ to n

$x[t] = i$ //第t个皇后放到第i列

 if **PLACE(t)**

BACKTRACK-NQUEEN1-REC(t+1)

PLACE(k)

 for $j = 1$ to $k-1$ //判断第k个皇后与前面k-1个皇后的是否冲突

 if **Math.abs(k-j) == Math.abs(x[j]-x[k]) || $x[j] == x[k]$**

 return false //相同列或者在相同斜线上

 return true

n皇后问题满排列树版本

问题解的形式：表示为n元组 $(x_1, x_2, \dots, x_n)$ 的形式，其中 $x_i (i=1, 2, \dots, n)$ 表示第i个皇后放置在**第i行第 x_i 列**；

显式约束：**n个皇后不同行且不同列**；

隐式约束：**n个皇后不在正反对角线上**；

解空间：根据显式约束，**第i(i=1,2,...,n)个皇后不能放置在前i-1个皇后所在的列，即仅有n-i+1个位置可供选择**。令 $S=\{1,2,\dots,n\}$ ，则 $x_i \in S - \{x_1, x_2, \dots, x_{i-1}\}$ 。显然，满足显式约束的n元组共有**n!**种，它构成n皇后问题的解空间。➔ **排列树**

BACKTRACK-NQUEEN2-REC(t)

if $t > n$

OUTPUT(x)

else

for $i = 1$ to n //n列

for $j = 1$ to $t-1$ //前面t-1个皇后

if $x[j] == i$ break

$x[t] = i$

if PLACE(t)

BACKTRACK-NQUEEN2-REC(t+1)

PLACE(k)

for $j = 1$ to $k-1$

if $\text{Math.abs}(k-j) == \text{Math.abs}(x[j]-x[k])$

return false

return true

算法	最坏时间复杂度	平均时间复杂度	最优时间复杂度	空间复杂度
NQUEEN1	$O(n^{(n+1)})$	$O(n!)$	$O(n)$	$O(n)$
NQUEEN2	$O(n! \times n)$	$O(k \times n!)$, $k < 1$	$O(n)$	$O(n)$

图的m着色问题

我赌他不考

分支限界

2025年5月22日 15:46

概念

基于广度优先策略

布线问题

```
GNode* findPath(g, a, b) {    // a为源点, b为目标点
    // 初始化
    g[a.row][a.col] = 0
    GNode* here = new GNode(a.row, a.col, NULL)
    Q = ∅    // 初始化活结点队列

    // BFS搜索
    while (here != NULL && here != b) {
        // 获取当前结点的所有子结点
        GNode[] childNodes = getChildNodes(here)

        // 遍历所有子结点
        for each n in childNodes {
            if (g[n.row][n.col] == -1) {    // 如果子结点未布线
                enqueue(Q, n)                // 加入队列
                n.pNode = here                // 设置父结点指针
                g[n.row][n.col] = g[here.row][here.col] + 1    // 更新距离
            }
        }

        // 取出下一个活结点
        here = dequeue(Q)
    }

    // 返回结果
    if (here != NULL && here == b) {
        output(here)    // 输出路径
        return here
    }

    return NULL    // 未找到路径
}
```

这个算法的时间复杂度是 $O(mn)$ ，其中 m 和 n 是网格的行数和列数，空间复杂度也是 $O(mn)$ 。

01背包问题普通队列法

```
int q_knapsack_01(int w[], int v[], int W) {
    // 初始化
    int i = 1;                // 当前物品编号
    int cw = 0;                // 当前重量
    int cp = 0;                // 当前价值
    int us = 0;                // 价值上限
```

```

int cw = 0;           // 当前重量
int cp = 0;           // 当前价值
int up = 0;           // 价值上界
int bestp = 0;        // 最优解价值
Queue q = Ø;          // 初始化队列
QNode stNode = NULL;  // 解路径记录

// 主循环
while (TRUE) {
    // 检查左孩子 (选择当前物品)
    if (cw + w[i] <= W) { // 重量约束满足
        // 更新最优解
        bestp = (cp + v[i] > bestp) ? cp + v[i] : bestp;

        // 创建左孩子结点并入队
        enqueue(q, new QNode(cw + w[i], cp + v[i], up, true, i + 1));
    }

    // 计算右孩子的价值上界
    up = bound(i + 1);

    // 检查右孩子 (不选择当前物品)
    if (up > bestp) { // 有希望得到更优解
        enqueue(q, new QNode(cw, cp, up, false, i + 1));
    }

    // 取出下一个活结点
    QNode node = dequeue(q);

    // 队列为空, 搜索结束
    if (node == NULL) {
        break;
    }

    // 更新当前状态
    if (node != NULL) {
        cw = node.cw;           // 当前重量
        cp = node.cp;           // 当前价值
        up = node.up;           // 价值上界
        i = node.level;         // 当前层次

        // 到达叶子结点, 记录解路径
        if (i == n + 1) {
            stNode = node.ptr; // 记录解路径起始结点
        }
    }
}

// 构造最优解向量
if (stNode != NULL) {
    for (int j = n; j >= 1; j--) {
        bestx[j] = stNode.isLeft ? 1 : 0; // 1表示选择, 0表示不选择
    }
}

```

```

if (stNode != NULL) {
    for (int j = n; j >= 1; j--) {
        bestx[j] = stNode.isLeft ? 1 : 0; // 1表示选择, 0表示不选择
        stNode = stNode.parent;         // 回溯到父结点
    }
    output(bestx); // 输出最优解
}

return bestp; // 返回最优解价值
}

```

计算上界限界函数需要 $O(n)$ 时间, 而最坏情况有 $2^{(n+1)}-2$ 个节点; 若对每个结点用**限界函数**判断, 则其时间复杂度为 $O(n*2^n)$ 。而算法中**时间复杂度**主要**依赖限界函数**, 则算法的时间复杂度为 $O(n*2^n)$ 。

空间复杂性: 最坏情况下需搜索 $2^{(n+1)}-2$ 个节点, 需 $O(2^n)$ 个空间存储节点, 则算法空间复杂性为 $O(2^n)$ 。

01背包问题优先队列法

优先队列式分支限界法求解0-1背包问题:

- 结点的优先级: $up = cp + r \cdot p$, **up越大, 优先级越高**。
- 针对当前扩展结点, 一次性生成**其所有孩子结点**, 并判断是否满足**约束条件和限界条件**。如果满足, 按设定的**优先级**将其**插入**到活结点表的相应位置; 反之, 舍弃。
- 从活结点表中**优先选取高优先级的结点进行扩展**。

```

Function PQ-BB-KNAPSACK-01(w, v, W)
// 输入: 物品重量 w[1..n], 价值 v[1..n], 背包容量 W
// 输出: bestx[1..n] 最优解, bestp 最优价值

n ← length(w)
cw ← 0           // 当前重量
cp ← 0           // 当前价值
bestp ← 0        // 当前已知的最优价值

```

```

cp ← 0           // 当前价值
bestp ← 0        // 当前已知的最优价值
up ← bound(1, cw, cp, w, v, W) // 当前结点的上界
Q ← empty priority queue (max-heap, sorted by up)
stNode ← NULL    // 最优解结点

INSERT(Q, new PQNode(cw, cp, up, NULL, 1, false)) // 初始根节点入队

while Q is not empty do
    node ← EXTRACT-MAX(Q)
    if node == NULL then break

    cw ← node.cw
    cp ← node.cp
    up ← node.up
    i ← node.level

    if i > n then
        if cp > bestp then
            bestp ← cp
            stNode ← node
        continue

    // 左子结点：选择第 i 个物品
    if cw + w[i] ≤ W then
        newcw ← cw + w[i]
        newcp ← cp + v[i]
        newup ← bound(i + 1, newcw, newcp, w, v, W)
        if newcp > bestp then bestp ← newcp
        INSERT(Q, new PQNode(newcw, newcp, newup, node, i + 1, true))

    // 右子结点：不选择第 i 个物品
    newup ← bound(i + 1, cw, cp, w, v, W)
    if newup > bestp then
        INSERT(Q, new PQNode(cw, cp, newup, node, i + 1, false))

// 回溯构造解向量
bestx[1..n] ← [0, ..., 0]
ptr ← stNode
while ptr ≠ NULL do
    if ptr.level > 1 then

```

```
while ptr ≠ NULL do
  if ptr.level > 1 then
    bestx[ptr.level - 1] ← ptr.isLeft ? 1 : 0
  ptr ← ptr.parent

OUTPUT(bestx, bestp)
```